



LET THERE BE LIGHT

SETUP WALKTHROUGH

NOW CUTS HOOK OVERHEAD 85%

"And God said, Let there be light: and there was light." — Genesis 1:3

The installer runs end-to-end. Prompt 0 picks the orchestrator (claude, cursor, cline, codex, aider or generic). Prompt 7 collects API keys with per-provider instructions and writes a `0600 .env`. Setup now also writes a `CLAUDE.md` doctrine file that is auto-loaded and prompt-cache friendly, so the routing map, keep-in-Claude rules, and 4-step pipeline never re-inject on every message. Multi-framework users get a lean, sibling-detected block that skips shared sections. Measured: **85% cut in hook overhead** vs the prior version on a 50-message session.

What changed since the last PDF: SessionStart shrank from ~1500 to ~250 bytes (routing map moved to CLAUDE.md). UserPromptSubmit dropped from ~820 to ~70 bytes per message. CLAUDE.md carries the doctrine once and Claude Code caches it for one hour. The pal-router skill gained 11 token-efficiency rules (auto-compress outputs over 5 KB, batch PAL calls, { claim, confidence, source_span } triples, in-conversation failure-map, diff-first, response terseness ladder). Sibling detection dedups doctrine when both sauron and let-there-be-light are installed. **PAL now installs from crowx01/pal-mcp-server** — the routing-enhanced fork.

1. The orchestrator choice

c	claude-code (default)	writes <code>~/.claude/settings.json</code> hooks + symlinks skills into <code>~/.claude/skills/</code>
r	cursor	writes <code>./.cursor/rules/let-there-be-light.mdc</code> with <code>alwaysApply:true</code> ; copies skills alongside
l	cline	writes <code>./.clinerules</code> (or <code>~/.clinerules</code>); copies skills alongside
x	codex-cli	writes <code>~/.codex/instructions.md</code> ; copies skills into <code>~/.codex/let-there-be-light-skills/</code>
a	aider	writes <code>./.aider.let-there-be-light.md</code> ; add to <code>.aider.conf.yml</code> as read:
g	generic	writes <code>./SYSTEM_PROMPT.let-there-be-light.md</code> you paste into any tool; copies skills alongside

2. The terminal session (Claude Code path)



interactive setup · from design to production

```
0. Which agent orchestrator do you use?
c) claude-code      (default)  full: hooks + Skill() invocations + delegate policy
r) cursor           writes .cursor/rules/Framework.mdc (alwaysApply)
l) cline            writes .clinerules with the delegate policy
x) codex-cli (OpenAI) writes ~/.codex/instructions.md
a) aider            writes .aider.conf.yml + Framework.md conventions
g) generic / other  writes SYSTEM_PROMPT.md you paste into any tool
✓ orchestrator: c

1. Where should it install?
g) global      → ~/.claude/settings.json      (loads every session, every project)
p) per-project → $PWD/.claude/settings.json   (only loads when Claude Code runs here)
s) skip        → print the settings JSON to stdout, do not write anything

2. Which skills should auto-load at session start?
These are invoked as the very first tool calls of every session.
Leave blank to skip a skill; type y to include it.

3. Which PAL models should be listed in the delegate-first policy?
Pick every model your PAL install actually has keys for.
Unselected models are dropped from the routing map so Claude never tries them.

Ready to write ~/.claude/settings.json
Existing file will be backed up with .bak.<timestamp> suffix.
✓ wrote ~/.claude/settings.json
✓ wrote CLAUDE.md doctrine: ~/.CLAUDE.md
✓ wrote env template: ~/.claude/.env.lttbl.example

7. Enter API keys now to complete installation?
Skip and edit .env.lttbl.example later, or enter them now to have a real
.env.lttbl written with 0600 permissions.

Next steps
1. Register PAL as an MCP server in ~/.claude.json (setup.sh does this for you
on the claude-code path):
  "mcpServers": { "pal": { "type": "stdio",
    "command": "/path/to/pal-mcp-server/.pal_venv/bin/python",
    "args": ["/path/to/pal-mcp-server/server.py"],
    "env": { ...keys... } } }
2. Source your API keys: source ~/.claude/.env.lttbl (after editing it)
3. Restart Claude Code.
4. On the next session start the auto-invoked skills fire immediately.

✓ let-there-be-light setup complete.
```

3. What Claude sees at the next session start (now lean)

The SessionStart hook fires before Claude responds. Its payload is injected into context. The full doctrine now lives in `CLAUDE.md`, so the hook is just a pointer.

SessionStart · additionalContext (249 bytes)

```
Doctrine + routing map are in CLAUDE.md at the project root (already loaded).
Preload skills now: `caveman` (full mode), `architect`, `test-strategist`,
`clean-code`, `fix-validator` (preload), `ship-checklist`, `incident-response`,
`retrospective`.
```

UserPromptSubmit · additionalContext (69 bytes, every message)

```
Doctrine active from CLAUDE.md. Delegate + failover. Batch PAL calls.
```

Why it is cheap: the routing map, keep-in-Claude list, failover doctrine, 4-step pipeline, and the 11 token-efficiency rules moved into `CLAUDE.md`. Claude Code auto-loads that file into the system prompt and caches it for one hour. On message 2 through N in the same session you pay **0 tokens** for it. The prior version re-injected all of it on every user message.

4. The CLAUDE.md doctrine file (4161 bytes, loaded once)

Setup writes this alongside `settings.json`. It sits inside a `<!-- BEGIN let-there-be-light doctrine -->` managed block so re-running `setup.sh` is idempotent. When both sauron and let-there-be-light are installed, the second one detects its sibling and writes a lean block that skips the shared sections.

```
<!-- BEGIN let-there-be-light doctrine (managed by setup.sh; regenerate to update) -->
# let-there-be-light doctrine

## Delegate-first routing
groq (gpt-oss-120b, ~500 rpm, 8000 tpm cap) for report writing, vuln explanations, and
skeptical validation. nemotron (nvidia, 1M ctx) for bulk reading of large files. grok
(x-ai, 2M ctx) for permissive high-context security reasoning when Gemini refuses. flash
(gemini-3.6-flash, 1M ctx) for fast structured extraction. or-free (OpenRouter meta-router,
200K ctx) as generalist fallback. pro (gemini-3.1-pro-preview) for deep-reasoning fallback
and adversarial debate via mcp_pal_challenge.

## Keep in Claude ONLY
- user-facing decisions
- exploitation choices, severity calls
- safety-boundary checks (no 3rd-party data, no destructive actions)
- side-effecting actions
- tool-sequence orchestration

## Failover
Any model refusal or error routes to the next model in the map. A classifier refusal is a
routing problem, not a stop.

## 4-step validation pipeline (per code fix before ship OR per confirmed finding)
A. Stress-test via fix-validator (or validator) skill.
B. Run the gap tests the validator flags.
C. Adversarial challenge via mcp_pal_challenge (pro primary, groq fallback).
D. Synthesize with confidence markers; groq drafts prose, Claude spot-checks
    low-confidence claims only.

## Token efficiency rules (this framework exists to cut Claude token cost)
- Any tool output over 5 KB routes through nemotron (bulk) or flash (structured) for
  summarization before Claude reads.
- Batch related PAL sub-tasks into one structured call, not N separate ones.

... [truncated for PDF; full file is 4161 bytes]
```

5. Files on disk after install

```
.claude/  
.claude/.env.lttbl.example  
.claude/settings.json  
CLAUDE.md
```

.env.lttbl.example (only the API keys your model picks need)

```
# let-there-be-light API keys - source from your shell rc, or export before Claude Code.  
# Do NOT commit the real values.  
export GEMINI_API_KEY=your-gemini-key  
export OPENROUTER_API_KEY=your-openrouter-key  
export CUSTOM_API_URL=https://api.groq.com/openai/v1  
export CUSTOM_API_KEY=your-groq-key
```

PAL server: github.com/crowx01/pal-mcp-server (routing-enhanced fork) · Repo: github.com/crowx01/let-there-be-light · Sibling: [crowx01/sauron](https://github.com/crowx01/sauron) (bug-bounty)